



Chapter 1: Introduction

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Challenges

写一个SQL：查询所有学生的编号，姓名和平均成绩，按总平均成绩降序排列



以下是查询所有学生编号、姓名和平均成绩，并按平均成绩降序排列的SQL语句：

Sql

深色版本 ∨ | 复制

SELECT

s.student_id AS 学生编号,
s.student_name AS 姓名,
AVG(sc.score) AS 平均成绩

FROM

students s
LEFT JOIN scores sc ON s.student_id = sc.student_id

GROUP BY

s.student_id, s.student_name

ORDER BY

平均成绩 DESC;



- 讨论：大模型时代为什么要学数据库？



Database Applications Examples

- Enterprise Information
 - Sales: customers, products, purchases
 - Accounting: payments, receipts, assets
 - Human Resources: Information about employees, salaries, payroll taxes.
- Manufacturing: management of production, inventory, orders, supply chain.
- Banking and finance
 - customer information, accounts, loans, and banking transactions.
 - Credit card transactions
 - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data)
- Universities: registration, grades

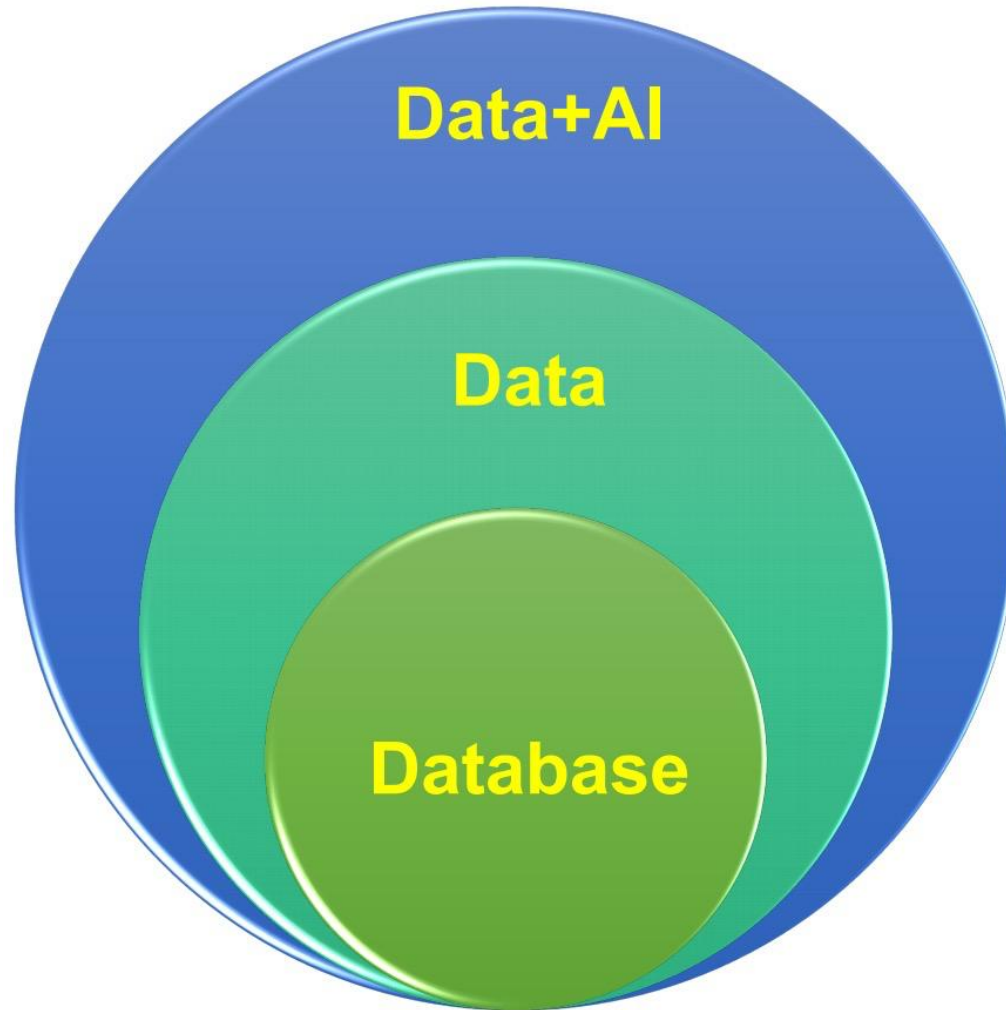


Database Applications Examples (Cont.)

- Airlines: reservations, schedules
- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
 - Online retailers: order tracking, customized recommendations
 - Online advertisements
- Document databases
- Navigation systems: For maintaining the locations of various places of interest along with the exact routes of roads, train systems, buses, etc.



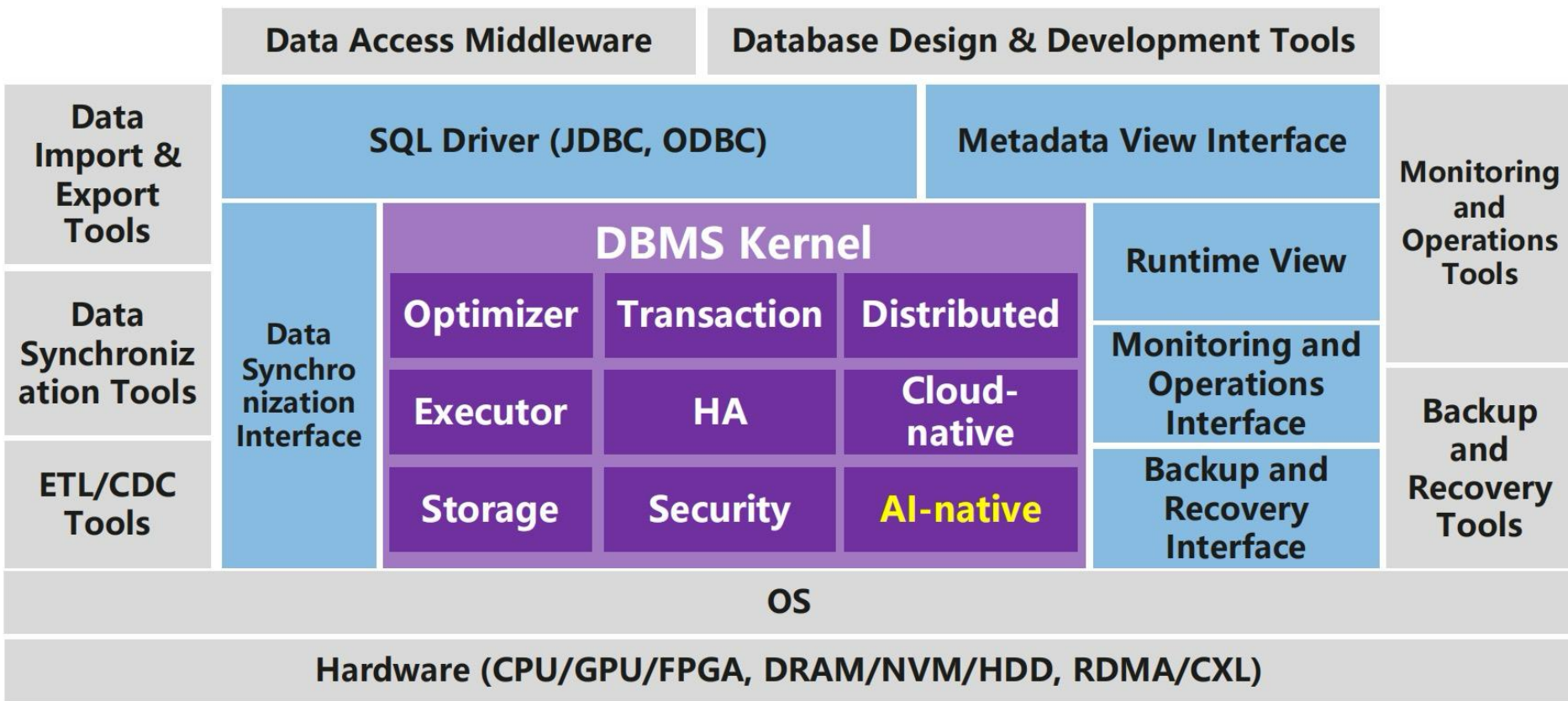
Database **to** Data Systems **to** Data + AI



NDBC 2025 Keynote on Data Agent.清华大学李国良教授



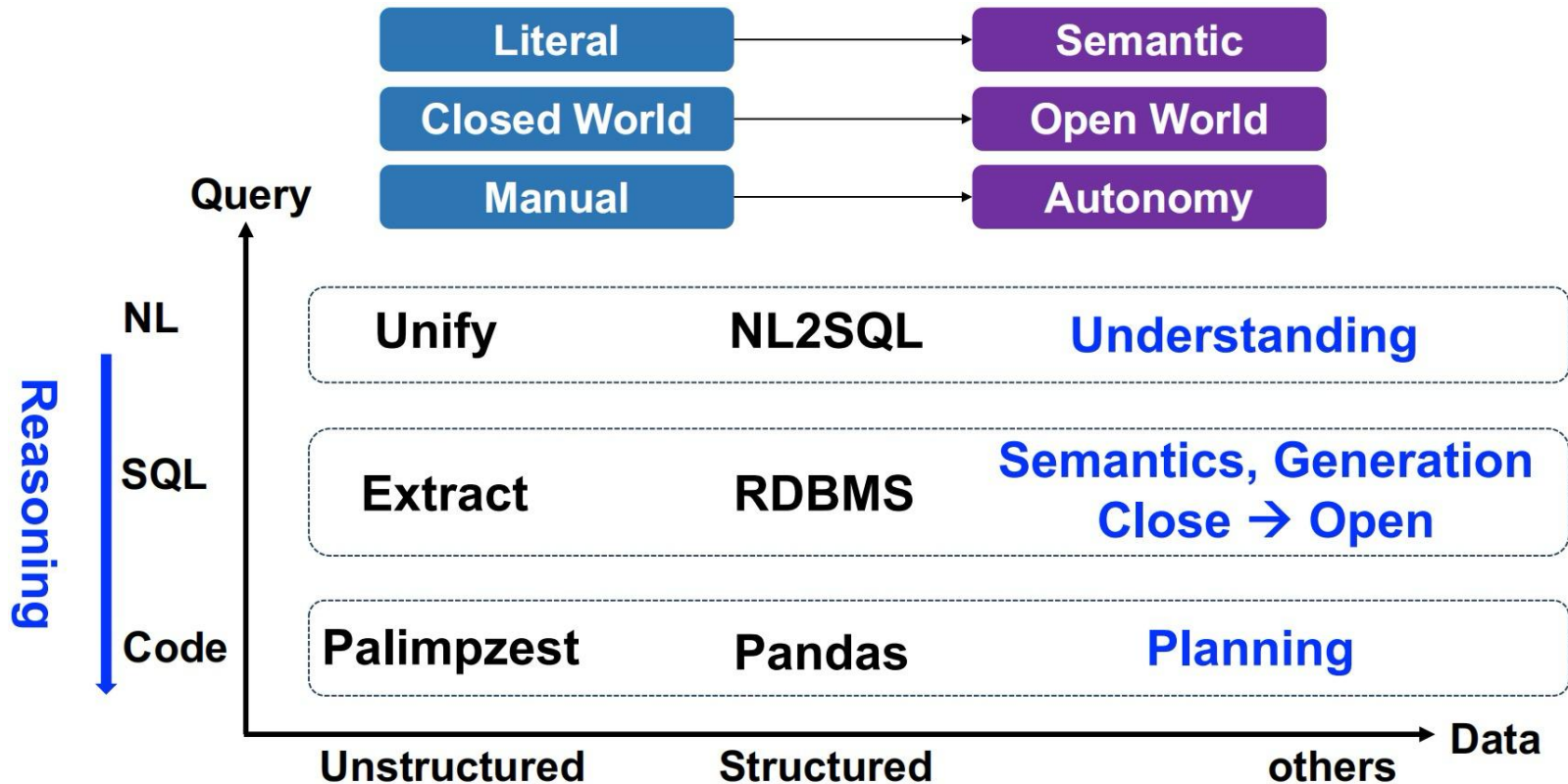
Database



NDBC 2025 Keynote on Data Agent.清华大学李国良教授



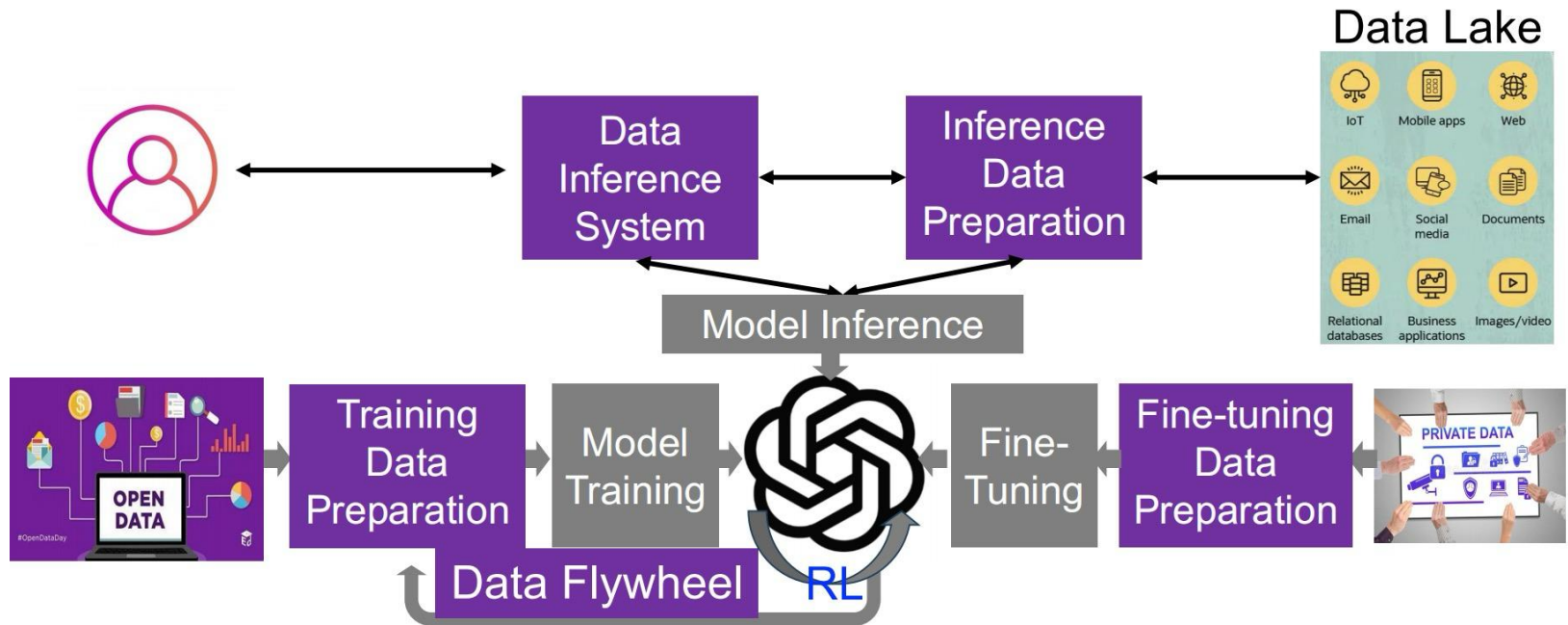
Database to Data Systems



NDBC 2025 Keynote on Data Agent.清华大学李国良教授



Data to Data + AI



NDBC 2025 Keynote on Data Agent.清华大学李国良教授



Outline

- **Data**
 - What is Data?
 - View of Data
- **Database Systems**



数据

■ 示例1：93

- 语义1： 学生某门课的成绩
- 语义2： 某人的体重
- 语义3： 计算机2025级学生人数
- 语义4： 请同学给出...



数据

■ 示例2：一幅黑白图像



- 数据——黑白点阵
- 语义——脸谱



数据

■ 数据

- 对现实世界中客观事物的符号表示
- 可以是数值数据，也可以是非数值数据，如声音、图像等
- 计算机中数据
 - **能输入计算机，并能为其处理的符号序列**
- 信息与数据的联系
 - 数据是信息的符号表示，或称载体
 - 信息是数据的内涵，是数据的语义解释
 - 数据是符号化的信息
 - 信息是语义化的数据



View of Data

- A major purpose of a database system is to provide users with an abstract view of the data.
 - **Data abstraction**
 - Hide the complexity of data structures to represent data in the database from users through several levels of data abstraction.
 - **Data models**
 - A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.



Data Abstraction

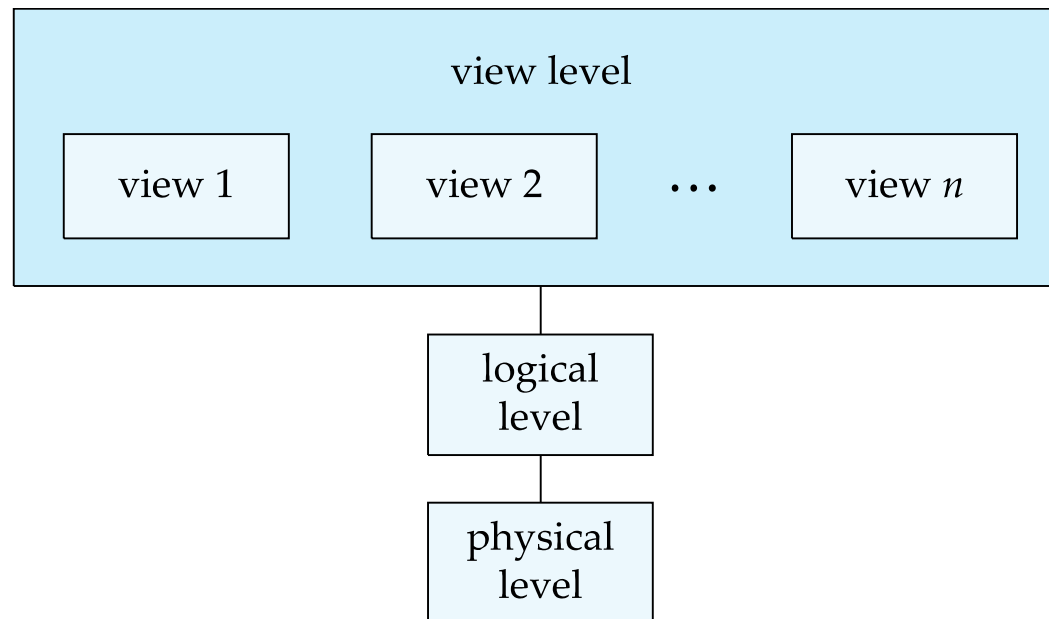
■ 数据抽象

- 数据库系统是面向计算机的，而应用是面向现实世界的，两个世界存在着很大差异，要直接将现实世界中的语义映射到计算机世界是十分困难的，因此引入一个信息世界作为现实世界通向计算机实现的桥梁
- 一方面，信息世界是对现实世界的抽象，从纷繁的现实世界中抽取出能反映现实本质的概念和基本关系;另一方面，信息世界中的概念和关系，要以一定的方式映射到计算机世界中去，在计算机系统上最终实现。信息世界起到了承上启下的作用



View of Data

An architecture for a database system:
three levels of data abstraction





Levels of Abstraction

- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

type *instructor* = **record**

```
ID : string;  
name : string;  
dept_name : string;  
salary : integer;  
end;
```

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.



Data Models (数据模型) 的三个层面

- 概念数据模型
- 逻辑数据模型
- 物理数据模型



概念数据模型

- 按用户的观点来对数据和信息建模
- 用于组织信息世界的概念，表现从现实世界中抽象出来的事物以及它们之间的联系
- 这类模型强调其语义表达能力，概念简单、清晰，易于用户理解
- 它是现实世界到信息世界的抽象，是用户与数据库设计人员之间进行交流的语言
- 如E-R模型



概念数据模型： E-R模型

- 历史
 - E - R 模型： Entity-Relationship Model
 - 1976年， P.P.S.Chen提出 E - R 模型， 用 E - R 图来描述概念模型
- 观点
 - 世界是由一组称作**实体**的基本对象和这些对象之间的**联系**构成的



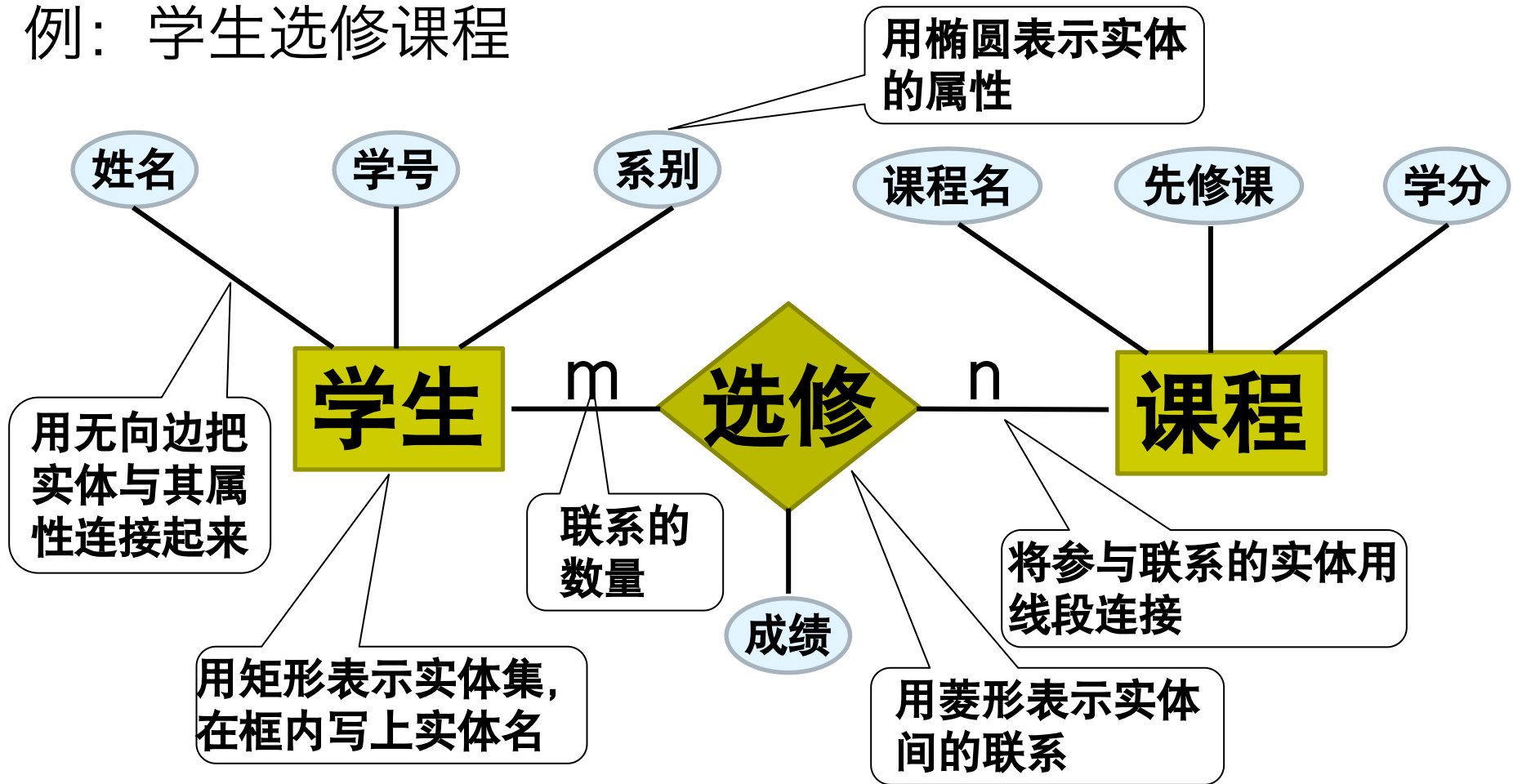
概念数据模型：E-R模型

- 实体(Entity)
 - 客观存在并可相互区分的事物叫实体
 - 如学生张三、工人李四、计算机系、数据库概论
- 属性(Attribute)
 - 实体所具有的某一特性
 - 一个实体可以由若干个属性来刻画
 - 例如，学生可由学号、姓名、年龄、系等组成
- 域(Domain)
 - 属性的取值范围
 - 例如，性别的域为（男、女），月份的域为 1 到 12 的整数



概念数据模型：E-R模型

例：学生选修课程





逻辑数据模型

- 从计算机实现的观点来对数据建模
- 是信息世界中的概念和联系在计算机世界中的表示方法
- 一般有严格的形式化定义，以便于在计算机上实现
- 如层次模型、网状模型、关系模型、面向对象模型



逻辑数据模型的三要素

■ 数据结构

- 描述系统的静态特性，即组成数据库的对象类型
- 数据本身
 - 类型、内容、性质。如网状模型中的数据项、记录，关系模型中的域、属性，关系等
- 数据之间的联系
 - 例如网状模型中的联系型，关系模型中的外码
- 在数据库系统中一般按数据结构的类型来命名数据模型



逻辑数据模型的三要素

■ 数据操作

- 描述系统的动态特性，即对数据库中对象的实例允许执行的操作的集合，包括操作及操作规则
- 一般有检索、更新（插入、删除、修改）操作
- 数据模型要定义操作含义、操作符号、操作规则，以及实现操作的语言

■ 数据的约束条件

- 数据的约束条件是完整性规则的集合，规定数据库状态及状态变化所应满足的条件，以保证数据的正确、有效、相容



逻辑数据模型：关系模型

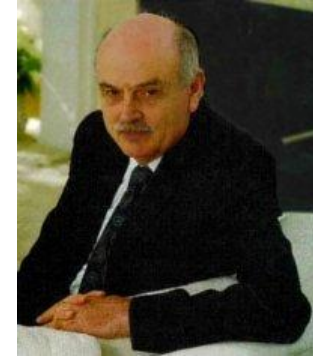
- 关系模型(relational model)
 - 用二维表来表示实体及其相互联系

元组	学号	姓名	年龄	性别	属性
	S01	张军	21	男	
	S02	李红	22	女	
	S03	王伟	19	男	

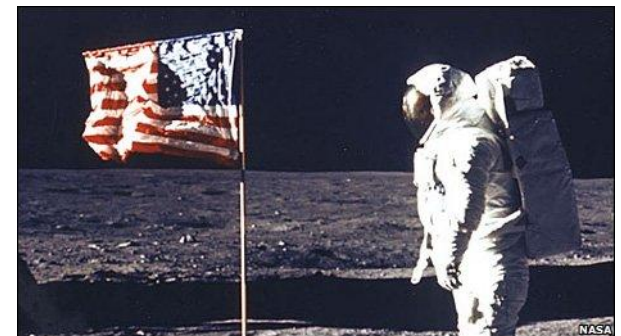


A Little History

- Relational model due to Edgar “Ted” Codd, a mathematician at IBM in 1970
 - A Relational Model of Data for Large Shared Data Banks. Communications of the ACM 13 (6): 377–387
- IBM didn’t want to use relational model (take money from IMS)
 - Apparently used in the moon landing...



Won Turing award 1981





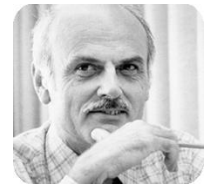
Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model

Columns

		-	

Rows



Ted Codd
Turing Award 1981



Outline

- Data
- **Database Systems**
 - Purpose of Database Systems
 - Database Users and Administrators
 - Database Languages
 - Database Design
 - Database Architecture
 - History of Database Systems



Database Systems

- DBMS contains information about a particular enterprise
 - Collection of interrelated data
 - Set of programs to access the data
 - An environment that is both *convenient* and *efficient* to use
- Database systems are used to manage collections of data that are:
 - Highly valuable
 - Relatively large
 - Accessed by multiple users and applications, often at the same time.
- A modern database system is a complex software system whose task is to **manage a large, complex collection of data**.
- Databases touch all aspects of our lives



Purpose of Database Systems

In the early days, database applications were built directly on top of file systems, which leads to:

- Data redundancy and inconsistency: data is stored in multiple file formats resulting in duplication of information in different files
- Difficulty in accessing data
 - Need to write a new program to carry out each new task
- Data isolation
 - Multiple files and formats
- Integrity problems
 - Integrity constraints (e.g., $\text{account balance} > 0$) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones



Purpose of Database Systems (Cont.)

- Atomicity of updates
 - Failures may leave database in an inconsistent state with partial updates carried out
 - Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
 - Concurrent access needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - Ex: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
 - Hard to provide user access to some, but not all, data

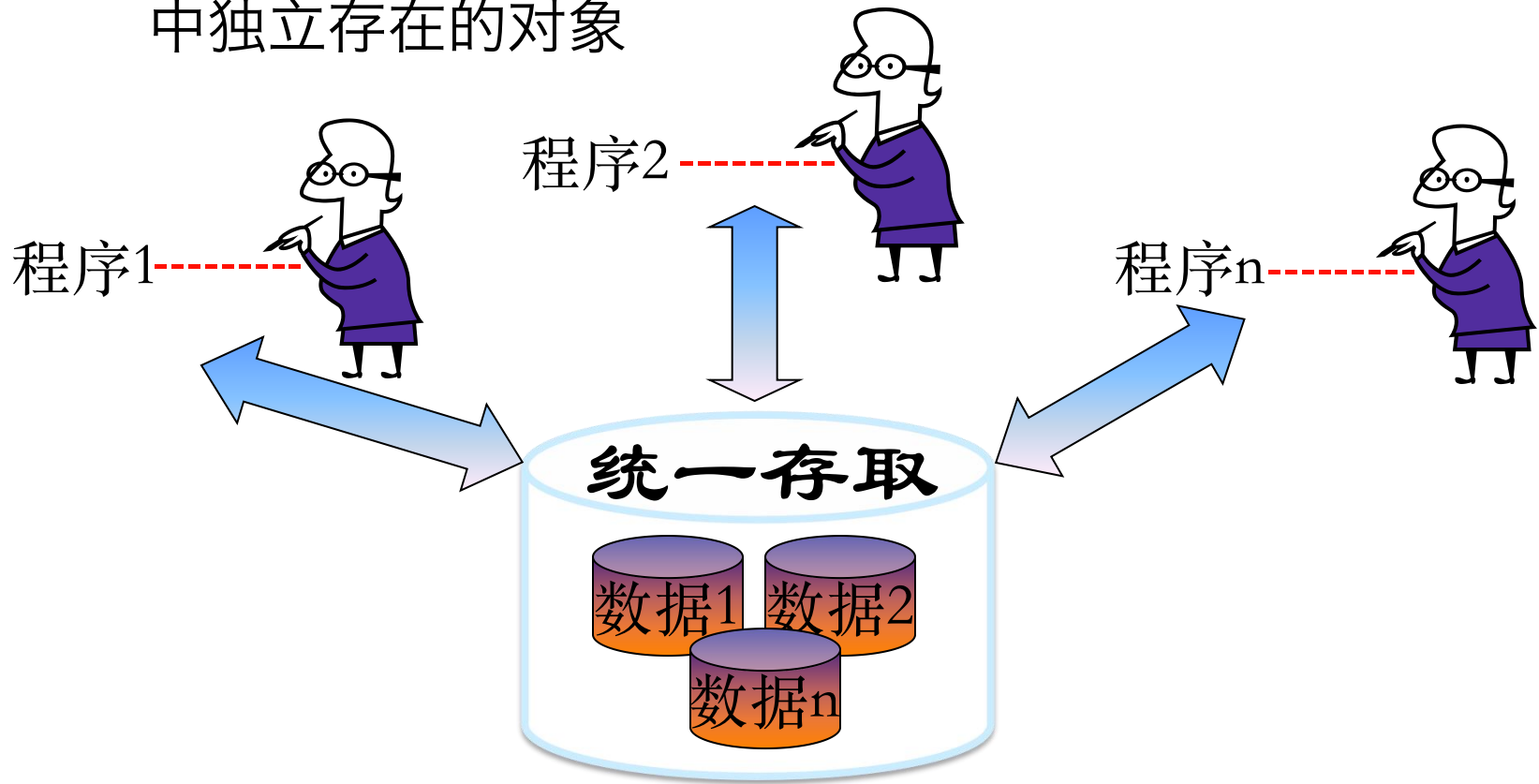
Database systems offer solutions to all the above problems



数据库系统阶段

■ 数据库观点

- 数据不是依赖于处理过程的附属品，而是现实世界中独立存在的对象





数据库系统的特点

- 具有较高的数据和程序的独立性
 - 把数据库的定义和描述从应用程序中分离出去
 - 数据描述是分级的（全局逻辑、局部逻辑、存储）
 - 数据的存取由系统管理，用户不必考虑存取路径等细节，从而简化了应用程序



概念

- 数据库系统DBS
 - 带有数据库的整个计算机系统，包括硬件、软件、数据、人员



数据库系统的主要成分

- 硬件 hardware
 - 大内存，放得下OS，DBMS核心，系统缓冲区，用户工作区等
 - 大容量、直接存取的外存设备
 - 作数据备份的磁带
- 软件 software
 - OS，DBMS，高级语言编译系统及其与数据库的接口，应用开发工具，应用系统
- 数据 data
 - 目标数据：数据本身
 - 描述数据：对数据的说明信息

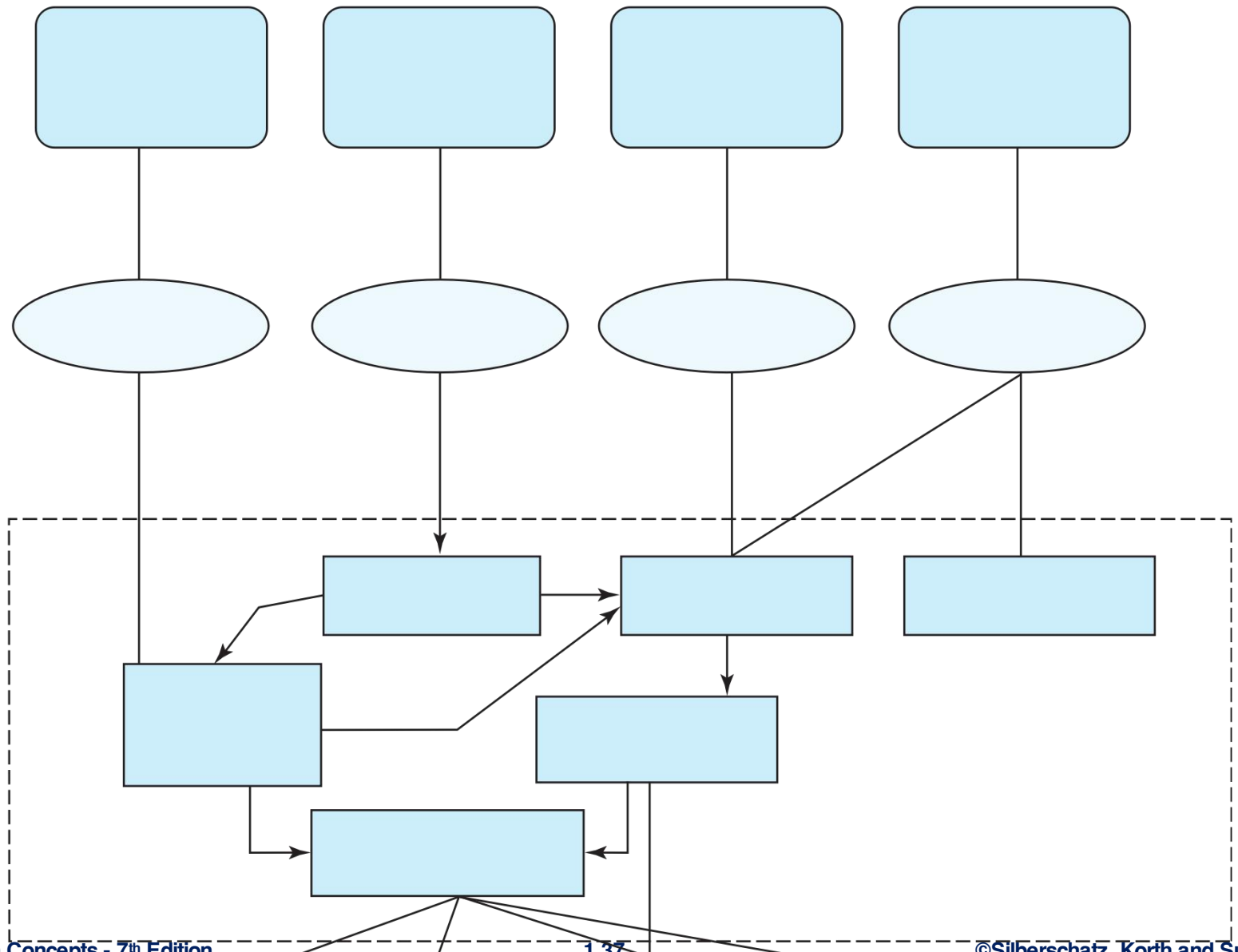


数据库系统的主要成分

- 用户
 - 最终用户(naive users)
 - 通过应用系统的用户接口（菜单等）使用数据库
 - 应用程序员(application programmers)
 - 基于外模式来编写应用程序
 - 系统分析员(system analysts)
 - 负责应用系统的需求分析和规范定义，确定系统的软硬件配置，参与数据库模式设计
 - 数据库管理员DBA(database administrator)
 - 负责数据库的全面管理和控制



Database Users





Database Administrator

A person who has central control over the system is called a **database administrator (DBA)**. Functions of a DBA include:

- Schema definition
- Storage structure and access-method definition
- Schema and physical-organization modification
- Granting of authorization for data access
- Routine maintenance
- Periodically backing up the database
- Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required
- Monitoring jobs running on the database



Database Languages

- 数据库定义功能
 - DDL语言 (Data-Definition Language)
- 数据存取功能
 - DML语言 (Data-manipulation language)
 - 对数据库进行检索、插入、修改、删除



Data Definition Language (DDL)

- Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a ***data dictionary***
- Data dictionary contains metadata (i.e., data about data)
 - Database schema
 - Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - Authorization
 - Who can access what



Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as query language
- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify what data are needed and how to get those data.
 - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs
- The portion of a DML that involves information retrieval is called a **query** language.



SQL Query Language

- SQL query language is nonprocedural. A query takes as input several tables (possibly only one) and always returns a single table.
- Example to find all instructors in Comp. Sci. dept

select *name*

from *instructor*

where *dept_name* = 'Comp. Sci.'

- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database



Database Access from Application Program

- Non-procedural query languages such as SQL are not as powerful as a universal Turing machine.
- SQL does not support actions such as input from users, output to displays, or communication over the network.
- Such computations and actions must be written in a **host language**, such as C/C++, Java or Python, with embedded SQL queries that access the data in the database.
- **Application programs** -- are programs that are used to interact with the database in this fashion.



Database Design

The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema.
Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database



Instances and Schemas (模式)

- Similar to types and variables in programming languages
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Physical schema** – the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable



Database Engine

- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be divided into
 - The storage manager,
 - The query processor component,
 - The transaction management component.



Storage Manager

- A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - Interaction with the OS file manager
 - Efficient storing, retrieving and updating of data
- The storage manager components include:
 - Authorization and integrity manager
 - Transaction manager
 - File manager
 - Buffer manager



Storage Manager (Cont.)

- The storage manager implements several data structures as part of the physical system implementation:
 - Data files -- store the database itself
 - Data dictionary -- stores metadata about the structure of the database, in particular the schema of the database.
 - Indices -- can provide fast access to data items. A database index provides pointers to those data items that hold a particular value.



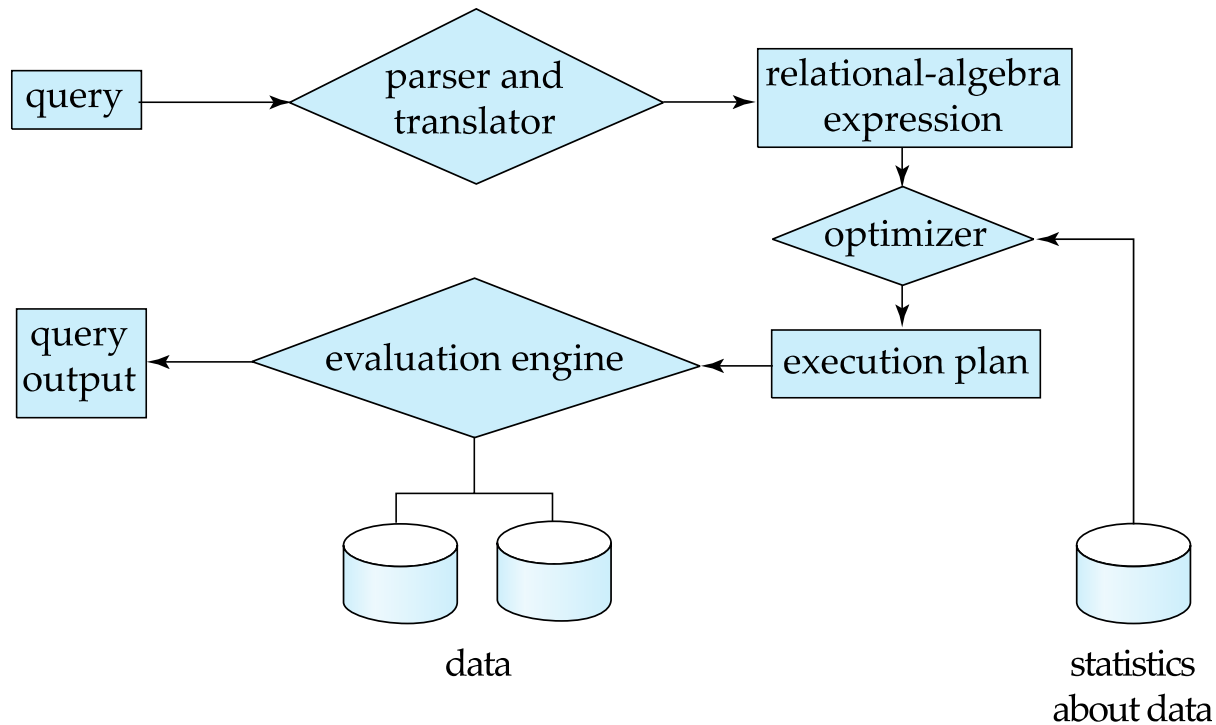
Query Processor

- The query processor components include:
 - DDL interpreter -- interprets DDL statements and records the definitions in the data dictionary.
 - DML compiler -- translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
 - The DML compiler performs query optimization; that is, it picks the lowest cost evaluation plan from among the various alternatives.
 - Query evaluation engine -- executes low-level instructions generated by the DML compiler.



Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Transaction Management

- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

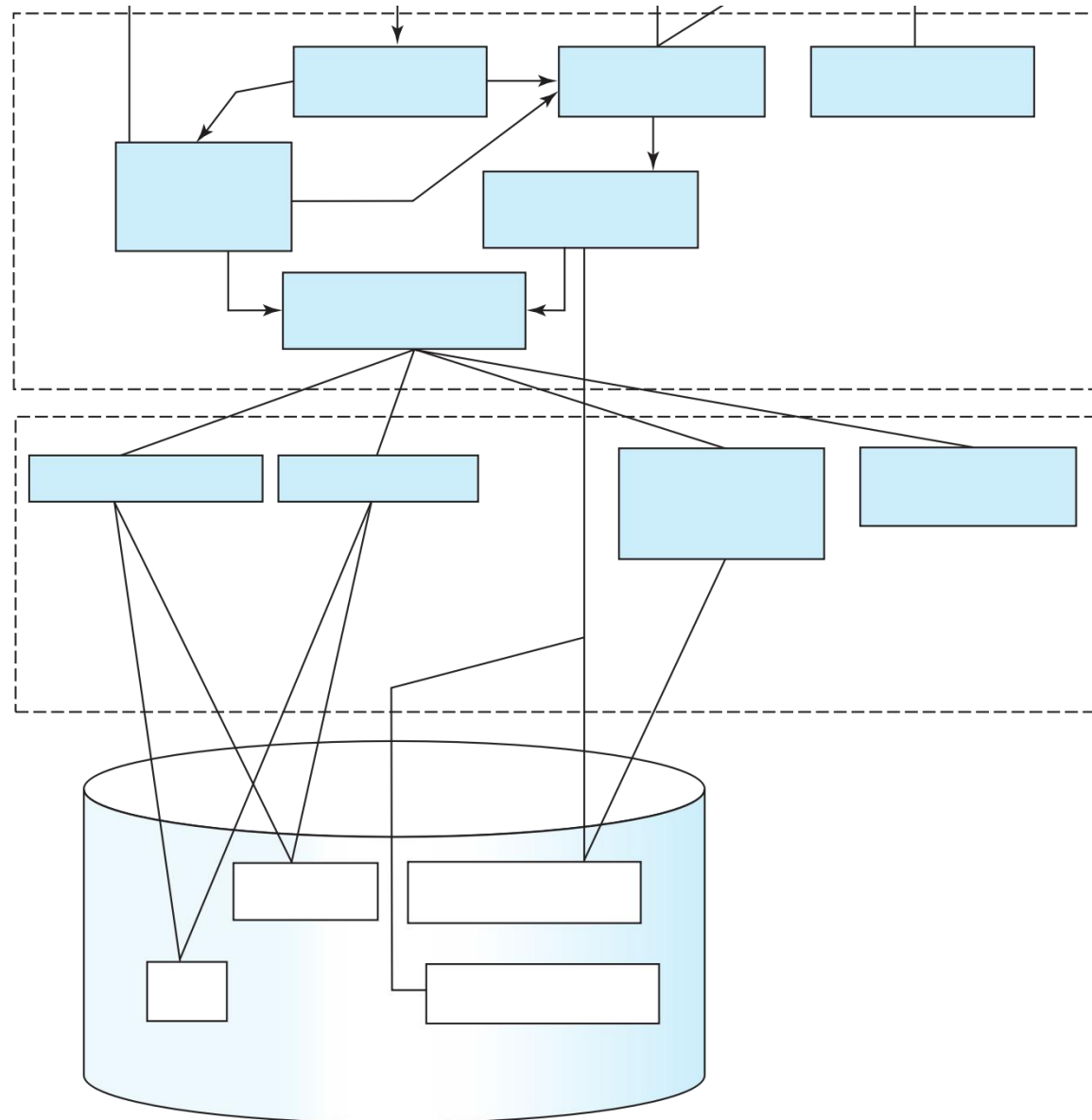


Database Architecture

- Centralized databases
 - One to a few cores, shared memory
- Client-server,
 - One server machine executes work on behalf of multiple client machines.
- Parallel databases
 - Many core shared memory
 - Shared disk
 - Shared nothing
- Distributed databases
 - Geographical distribution
 - Schema/data heterogeneity



Database Architecture (Centralized/Shared-Memory)





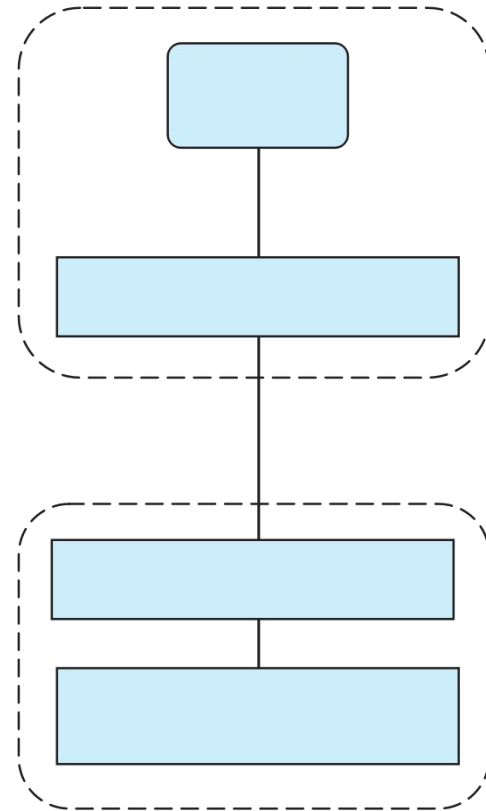
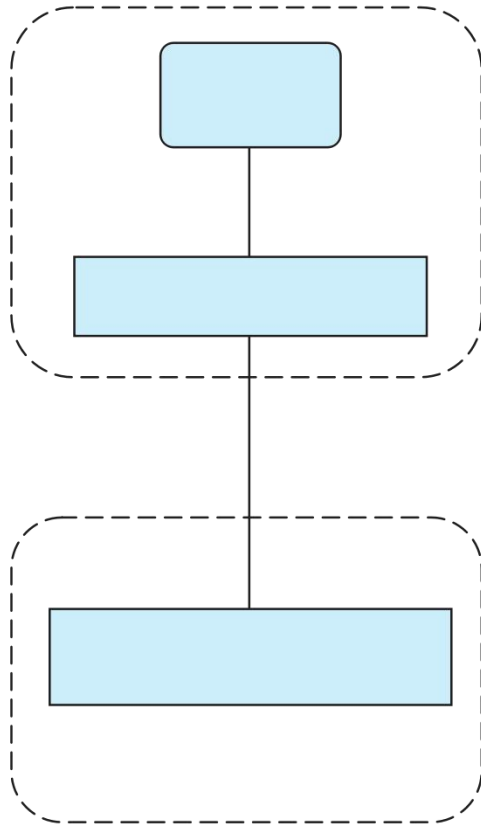
Database Applications

Database applications are usually partitioned into two or three parts

- Two-tier architecture -- the application resides at the client machine, where it invokes database system functionality at the server machine
- Three-tier architecture -- the client machine acts as a front end and does not contain any direct database calls.
 - The client end communicates with an application server, usually through a forms interface.
 - The application server in turn communicates with a database system to access data.



Two-tier and three-tier architectures





History of Database Systems

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - Would win the ACM Turing Award for this work
 - IBM Research begins System R prototype
 - UC Berkeley (Michael Stonebraker) begins Ingres prototype
 - Oracle releases first commercial relational database
 - High-performance (for the era) transaction processing



History of Database Systems (Cont.)

- 1980s:
 - Research relational prototypes evolve into commercial systems
 - SQL becomes industrial standard
 - Parallel and distributed database systems
 - Wisconsin, IBM, Teradata
 - Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce



History of Database Systems (Cont.)

- 2000s
 - Big data storage systems
 - Google BigTable, Yahoo PNuts, Amazon,
 - “NoSQL” systems.
 - Big data analysis: beyond SQL
 - Map reduce and friends
- 2010s
 - SQL reloaded
 - SQL front end to Map Reduce systems
 - Massively parallel database systems
 - Multi-core main-memory databases



End of Chapter 1